

Contents

guide Direct Preference Optimization: Your Language Model is Secretly a Reward Model 1

- 0. 这篇论文到底在改写什么 1
- 1. 原论文结构地图 2
- 2. 回到当时的语境: 为什么 RLHF 让人又爱又怕 2
- 3. RLHF baseline: DPO 到底省掉了什么 2
- 4. Figure 1 的核心图 3
- 5. DPO 的关键推导 4
- 6. DPO loss: 公式逐项拆开 4
- 7. 张量级别怎么实现 5
- 8. DPO update 到底在做什么 5
- 9. “语言模型偷偷是 reward model” 是什么意思 5
- 10. 和 PPO/RLHF 的关系 6
- 11. 实验设计: 论文到底证明了什么 6
- 12. DPO 和后续偏好优化家族 7
- 13. 和本仓库代码怎么对上 7
- 14. 30 分钟本地实验 7
- 15. 常见误区 8
- 16. 现代意义 8
- 17. 闭卷掌握检查 8
- 18. 用 AI agent 学这篇的正确方式 9

guide_Direct Preference Optimization: Your Language Model is Secretly a Reward Model

原论文: Direct Preference Optimization: Your Language Model is Secretly a Reward Model

本地原文 PDF: [learning/dpo-family/paper/01_direct_preference_optimization.pdf](#)

作者: Rafailov et al.

年份: 2023, arXiv v3 2024-07-29

类型: paper

0. 这篇论文到底在改写什么

DPO 的核心主张是: 标准 RLHF 里 “先训练 reward model, 再用 PPO 做强化学习” 的两阶段偏好优化, 可以改写成直接训练 policy 的二分类损失。论文标题里的 “Your Language Model is Secretly a Reward Model” 不是比喻, 而是一个数学重参数化: policy 相对 reference policy 的 log-ratio 可以被解释成一个隐式 reward。

这篇论文的历史位置很重要。InstructGPT 之后, RLHF 三段式成为对齐主流:

- 1. SFT: 用高质量示范数据把预训练模型变成会听指令的初始策略。
- 2. RM: 收集人类对两个回答的偏好, 训练 reward model。
- 3. PPO: 用 reward model 给采样回答打分, 再用强化学习更新 policy, 同时加 KL 约束防止模型跑偏。

这个流程有效，但工程上重：要训练多个模型，要采样，要调 PPO，要处理 reward hacking 和 KL 崩坏。DPO 说：如果我们接受 Bradley-Terry 偏好模型和 KL-constrained reward maximization 这个理论框架，那么不必显式学 reward，也不必跑 PPO。偏好对 (prompt, chosen, rejected) 本身就足以定义一个可微的 policy loss。

读这篇要抓住三层变化：

1. 从显式 reward model 变成隐式 reward。
2. 从 RL update 变成 supervised classification-style update。
3. 从 “优化一个 learned reward” 变成 “让 chosen/rejected 的相对 log-prob 符合偏好概率”。

1. 原论文结构地图

建议按下面顺序读原文：

1. Abstract/Introduction: 先看作者如何描述 RLHF 的复杂和不稳定。
2. Figure 1: 这是全篇最重要的图，比较 RLHF pipeline 和 DPO pipeline。
3. Section 3: 复习 RLHF 三阶段、Bradley-Terry reward model、KL-constrained RL objective。
4. Section 4: 推导 DPO objective。这里是论文主菜。
5. “What does the DPO update do?”：看 gradient 解释，理解为什么不是 naive chosen/rejected likelihood ratio。
6. Section 5: “Your Language Model Is Secretly a Reward Model”，理解 reward equivalence class。
7. Experiments: sentiment control、summarization、single-turn dialogue。
8. Appendix/ablation: 特别看 naive objective 为什么会让模型退化。

如果时间紧，第 3-4 节必须精读。DPO 不是经验 trick，它的说服力来自对 RLHF objective 的重写。

2. 回到当时的语境：为什么 RLHF 让人又爱又怕

RLHF 在 InstructGPT、ChatGPT 之后被证明很有用，因为它能把 “预测互联网文本” 的语言模型，调成 “更符合人类偏好” 的 assistant。但 RLHF 训练本身有几个痛点：

- reward model 要单独训练，而且 reward 只在其训练分布附近可信。
- PPO 对超参敏感，尤其是 KL 系数、reward scale、采样策略、batch size。
- 语言是离散 token，不能像连续控制那样直接对 action 平滑优化。
- RL 训练过程需要不断从当前 policy 采样，成本更高。
- reward model 可能被 policy exploit，出现 reward hacking。

所以 DPO 的 motivation 很朴素：既然偏好数据本来就是 (chosen > rejected)，能不能直接让 policy 在这些 pair 上学会偏好，而不要中间 reward model 和 PPO loop？

这不是简单的 “用 pairwise loss 训练模型”。如果只是让 chosen 概率上升、rejected 概率下降，模型可能退化，因为它没有 reference policy 的锚点，也没有区分 “本来就很容易的样本” 和 “当前排序错得很严重的样本”。DPO 的理论价值就在于：它从 KL-constrained RLHF objective 推出一个带 reference 的偏好损失。

3. RLHF baseline: DPO 到底省掉了什么

原论文先复习经典 RLHF。给定 prompt x ，SFT model 生成两个回答 y_1, y_2 ，人类标注哪个更好。记 preferred answer 为 y_w ，dispreferred answer 为 y_l 。

经典 reward model 用 Bradley-Terry 模型：

$$P(y_w > y_l \mid x) = \text{sigmoid}(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))$$

训练 reward model 就是让 preferred 的 reward 比 rejected 的 reward 高。

之后 PPO 阶段优化:

$$\max_{\pi} E_{x, y \sim \pi} [r_{\phi}(x, y) - \beta * KL(\pi(y|x) \parallel \pi_{\text{ref}}(y|x))]$$

这里 π_{ref} 通常是 SFT model。KL 项有两个作用:

1. 防止 policy 离开 reward model 可信的区域。
2. 防止模型为了高 reward 变得模式坍塌，只输出少数奇怪高分回答。

DPO 保留这个 KL-regularized RLHF 的理论目标，但换掉求解方式。

4. Figure 1 的核心图

可以把 Figure 1 用文字画成这样:

```

Classic RLHF
-----
pretrained LM
  |
  v
SFT policy pi_sft
  |
  +--> sample response pairs --> human preference data
                                   |
                                   v
                                   train reward model r_phi
                                   |
                                   v
                                   PPO optimize policy pi_theta
                                   reward - beta * KL(pi || pi_ref)

DPO
---
SFT/reference policy pi_ref
  |
preference pairs (x, y_w, y_l)
  |
  v
direct binary classification loss on policy log-ratios
  |
  v
aligned policy pi_theta

```

这张图要读出一个关键交换: DPO 不是不要偏好模型，而是把偏好模型的参数从显式 $r_{\phi}(x, y)$ 换成由 π_{theta} 和 π_{ref} 决定的隐式 reward。

5. DPO 的关键推导

从 KL-constrained reward maximization 出发, 给定任意 reward $r(x,y)$, 最优 policy 有 closed form:

$$\pi_r(y|x) = 1 / Z(x) * \pi_{\text{ref}}(y|x) * \exp(r(x,y) / \beta)$$

这里:

- $\pi_{\text{ref}}(y|x)$ 是 reference policy。
- β 控制 KL 约束强度。
- $Z(x)$ 是 partition function, 保证概率归一化。

这个式子可以反过来解出 reward:

$$r(x,y) = \beta * \log(\pi_r(y|x) / \pi_{\text{ref}}(y|x)) + \beta * \log Z(x)$$

看起来 $Z(x)$ 很麻烦, 但 Bradley-Terry 偏好模型只关心两个回答的 reward 差:

$$r(x,y_w) - r(x,y_l)$$

同一个 prompt x 下, $\beta * \log Z(x)$ 会抵消。于是偏好概率可以直接写成 policy/reference log-ratio 的差。

这就是 DPO 的灵魂: 通过 reward difference, 绕开 partition function, 也绕开显式 reward model。

6. DPO loss: 公式逐项拆开

DPO 的训练样本是:

(x, y_w, y_l)

x : prompt
 y_w : preferred/chosen response
 y_l : dispreferred/rejected response

对每个回答, 计算当前 policy 和 reference policy 的序列 log-prob:

$\log \pi_{\theta}(y_w | x)$
 $\log \pi_{\text{ref}}(y_w | x)$
 $\log \pi_{\theta}(y_l | x)$
 $\log \pi_{\text{ref}}(y_l | x)$

构造 margin:

margin =
[$\log \pi_{\theta}(y_w|x) - \log \pi_{\text{ref}}(y_w|x)$]
- [$\log \pi_{\theta}(y_l|x) - \log \pi_{\text{ref}}(y_l|x)$]

loss:

$L_{\text{DPO}} = -\log \text{sigmoid}(\beta * \text{margin})$

直觉:

- 如果 chosen 相对 reference 被当前 policy 提高了, rejected 相对 reference 被降低了, margin 就大, loss 小。
- 如果当前 policy 反而更偏向 rejected, margin 小甚至为负, loss 大。
- reference log-prob 是锚点, 防止模型只是一味抬高 chosen 或压低 rejected。
- β 控制偏好约束强度。 β 太小, 更新弱; β 太大, 可能过度追偏好对, 偏离 reference。

7. 张量级别怎么实现

一个 batch 的形状可以写成:

```
prompts:          [B]
chosen_ids:        [B, T_chosen]
rejected_ids:      [B, T_rejected]
chosen_mask:       [B, T_chosen]
rejected_mask:     [B, T_rejected]
```

```
policy_logits:    [B, T, vocab]
reference_logits: [B, T, vocab]
```

```
seq_logp_chosen:  [B]
seq_logp_reject:  [B]
loss:             scalar
```

序列 log-prob 通常是 token log-prob 在 response mask 上求和:

```
def sequence_logprob(logits, labels, mask):
    logp = logits.log_softmax(dim=-1)
    token_logp = logp.gather(-1, labels[... , None]).squeeze(-1)
    return (token_logp * mask).sum(dim=-1)
```

然后:

```
def dpo_loss(pi_c, pi_r, ref_c, ref_r, beta=0.1):
    chosen_adv = pi_c - ref_c
    reject_adv = pi_r - ref_r
    margin = chosen_adv - reject_adv
    return -torch.logsigmoid(beta * margin).mean()
```

注意这里的 `pi_c/ref_c` 都是序列 log-prob, 不是单个 token 概率。实现时最容易错的是 `mask`: prompt token 不应该计入 response log-prob, 否则模型会在不可训练的上下文部分获得虚假 margin。

8. DPO update 到底在做什么

论文专门分析了 DPO gradient。直观说, DPO 会:

1. 增加 preferred response 的 log-prob。
2. 降低 dispreferred response 的 log-prob。
3. 对 “当前隐式 reward 排错了” 的样本给更大权重。

这第三点很关键。DPO 不是所有 pair 都等权硬推。如果当前 policy 已经非常偏向 chosen, loss 梯度会小; 如果当前 policy 还偏向 rejected, 梯度会大。

这就是论文里说 naive probability ratio objective 会退化的原因之一。没有这个动态 weighting, 模型容易用粗暴方式提高 chosen/rejected 差距, 损伤语言质量。

9. “语言模型偷偷是 reward model” 是什么意思

Section 5 讨论 reward equivalence class。偏好模型只关心 reward 差, 因此如果两个 reward function 相差一个只依赖 prompt 的函数 $f(x)$:

$$r'(x, y) = r(x, y) + f(x)$$

它们对同一个 prompt 下的两个回答给出的偏好概率相同。也就是说，reward 的绝对值不重要，重要的是同一 prompt 下回答之间的相对差。

DPO 定义的隐式 reward 是：

```
r_hat_theta(x,y)
= beta * log(pi_theta(y|x) / pi_ref(y|x))
```

这个 reward 不需要单独的 reward head。当前 policy 相对 reference 更愿意生成某个回答，就等价于给它更高的隐式 reward。

这句话也有边界：它建立在 Bradley-Terry/Plackett-Luce 类偏好模型和 KL-constrained objective 的理论框架里。不是说任意语言模型天然就是可靠 reward model。

10. 和 PPO/RLHF 的关系

DPO 和 PPO-RLHF 的关系可以这样理解：

PPO-RLHF：

```
learn r_phi from preferences
sample y from current policy
optimize reward - KL with RL
```

DPO：

```
rewrite reward in terms of policy/reference log-ratio
optimize preference likelihood directly on offline pairs
```

所以 DPO 的优点：

- 不需要在线 RL sampling。
- 不需要训练显式 reward model。
- 实现接近 supervised fine-tuning。
- 超参更少，训练更稳定。

代价和风险：

- 依赖偏好数据质量。
- 主要是 offline pairwise preference optimization。
- 如果 chosen/rejected 数据分布窄，policy 学到的偏好也窄。
- beta、reference model、response length normalization 仍然会显著影响结果。

11. 实验设计：论文到底证明了什么

论文做了三类任务：

1. Sentiment modulation。
2. Summarization。
3. Single-turn dialogue。

作者比较 DPO 和 PPO-based RLHF 等方法，并报告 DPO 在这些任务上可以达到或超过 PPO，尤其在 sentiment control 上超过 PPO；在 summarization 和 single-turn dialogue 上匹配或改善 response quality，同时实现和训练更简单。

读实验时要分三层：

- **主结果：**DPO 不是只在 toy setting 有效，能处理真实 LM preference data。

- **工程结果:** DPO 不需要 PPO 的复杂采样和调参, 训练更轻。
- **机制结果:** naive variant 会退化, 说明 DPO 的 reference/weighting 设计不是装饰。

这篇论文没有证明:

- DPO 永远优于 PPO。
- DPO 自动解决 reward hacking。
- DPO 不需要高质量偏好数据。
- DPO 对多轮复杂 agent 或 long-horizon tasks 一定足够。
- beta/reference 的选择不重要。

12. DPO 和后续偏好优化家族

DPO 之后很多方法都在改它的某个假设:

- IPO: 重新分析 preference objective, 缓解 DPO 过拟合偏好强度的问题。
- KTO: 不要求 paired preference, 用好/坏反馈构造 Kahneman-Tversky 风格目标。
- ORPO: 试图把 SFT 和 preference alignment 合并。
- SimPO: 简化 reference 依赖, 关注 chosen/rejected 的平均 log-prob margin。
- CPO/DPOP/RainbowPO: 从稳定性、分布偏移、组合目标等角度继续修。

所以 DPO 是母式, 不是终点。读懂它, 后面所有 *_minimal.py 的损失函数都能放到一张地图上。

13. 和本仓库代码怎么对上

优先打开:

- learning/dpo-family/lectures/01-dpo.md
- learning/dpo-family/src/dpo_minimal.py
- learning/dpo-family/src/ipo_minimal.py
- learning/dpo-family/src/kto_minimal.py
- learning/dpo-family/src/tests/test_dpo_loss_equivalence.py

建议你在代码里确认三件事:

1. chosen/rejected 的 log-prob 是怎么从 token logits 聚合成 sequence log-prob 的。
2. reference model 的 log-prob 是否 detach/frozen。
3. beta 变化时, loss 和 gradient magnitude 如何变化。

14. 30 分钟本地实验

实验目标: 验证 beta 和 margin 对 DPO loss 的影响。

```
import torch
import torch.nn.functional as F

pi_c = torch.tensor([-8.0, -6.0, -5.0])
pi_r = torch.tensor([-10.0, -5.5, -4.0])
ref_c = torch.tensor([-9.0, -6.5, -5.2])
ref_r = torch.tensor([-9.5, -6.0, -4.5])

for beta in [0.05, 0.1, 0.5, 1.0]:
    margin = (pi_c - ref_c) - (pi_r - ref_r)
```

```
loss = -F.logsigmoid(beta * margin)
print(beta, margin.tolist(), loss.tolist())
```

观察:

- margin 越大, loss 越小。
- beta 越大, 同样 margin 的惩罚更陡。
- margin 为负的样本贡献更大, 代表当前 policy 相对 reference 更偏向 rejected。

把这个实验和 `dpo_minimal.py` 对上, 你就能理解 DPO loss 不是魔法, 只是一个带 reference 的 pairwise logistic loss。

15. 常见误区

- 误区 1: DPO 等于 SFT on chosen。
 - 错。DPO 同时看 chosen 和 rejected, 并且看相对 reference 的 log-ratio。
- 误区 2: DPO 不需要 reference model。
 - 错。reference 是 KL 约束在 loss 里的影子。
- 误区 3: DPO 不属于 RLHF。
 - 不准确。它不跑 RL loop, 但它优化的是和 KL-constrained RLHF 对应的偏好目标。
- 误区 4: beta 越大越好。
 - 错。beta 是偏好强度/偏离 reference 的旋钮, 过大可能损伤语言质量。
- 误区 5: DPO 解决了所有 alignment 问题。
 - 错。DPO 只是偏好优化算法, 数据、policy coverage、安全边界仍然重要。

16. 现代意义

DPO 今天仍然重要, 因为它把 alignment training 的门槛降得很低。很多团队没有条件稳定跑 PPO, 但可以跑 DPO/SimPO/ORPO 类 offline preference fine-tuning。它也是理解 “RL-free alignment” 争论的入口。

但从 2025 之后看, DPO 也有边界。Reasoning RL、tool-use agent、多轮任务、可验证奖励场景里, 单步偏好对不一定覆盖 long-horizon behavior。R1/GRPO/DAPO 重新把 RL 带回中心, 说明 DPO 和 RLVR 是互补路线: DPO 擅长偏好风格和回答质量对齐, RLVR 擅长可验证任务上的探索和强化。

17. 闭卷掌握检查

你应该能不看笔记回答:

1. 经典 RLHF 三阶段分别是什么, DPO 省掉了哪两块复杂度?
2. Bradley-Terry 模型为什么只关心 reward difference?
3. KL-constrained objective 的 closed-form optimal policy 长什么样?
4. 为什么 partition function $Z(x)$ 在偏好差里会抵消?
5. DPO loss 里的 $\log \pi_{\theta} - \log \pi_{\text{ref}}$ 表示什么?
6. beta 变大时, 训练行为可能怎么变?
7. DPO 和 “只提高 chosen 概率” 的 naive objective 有什么区别?
8. 为什么说语言模型相对 reference 的 log-ratio 可以看成 implicit reward?
9. DPO 实验主要证明了什么, 没有证明什么?
10. 如果在本仓库写 DPO 单元测试, 你会构造什么 chosen/rejected log-prob?

18. 用 AI agent 学这篇的正确方式

我正在学 DPO。请你不要直接总结论文。

请先让我手写 DPO loss，并解释每个 log-prob 项来自 policy 还是 reference。

然后用 5 个具体数字构造 chosen/rejected 的例子，让我判断 loss 大小。

如果我把 DPO 说成 SFT on chosen，请纠正我，并要求我解释 Bradley-Terry 和 KL reference 的作用。

最后让我把 DPO 和 PPO-RLHF 画成两张流程图。

真正掌握 DPO 的标志是：你能从 RLHF objective 推到 DPO loss 的直觉，能在代码里正确处理 response mask 和 reference log-prob，并能说清楚它为什么简单、为什么有效、又为什么不是 alignment 的终点。